

da classe de palavra atribuída, a palavra pode receber *features* de adjetivos ou *features* de verbos. Se em um texto clínico, encontramos a frase “Paciente lúcido e orientado”, interpretamos “lúcido” e “orientado” como adjetivos que qualificam o substantivo “paciente” e constroem significados do estado de saúde do mesmo. Nesse caso, “orientado” é classificado como adjetivo e recebe as *features* de gênero e número. Já em “Paciente orientado a procurar fisioterapia”, “orientado” é classificado como particípio passado do verbo “orientar”, em uma oração passiva sem explicitação do auxiliar “ser” e sem realização de uma função que indica o Papel Temático de Agente da ação, o qual no contexto pode ser interpretado como o profissional da saúde que realizou o atendimento. Neste caso, “orientado” recebe *features* da classe verbo e também da classe adjetivo, uma vez que em português os particípios passados concordam em gênero e número com os substantivos que realizam o sujeito da oração: “O paciente foi orientado”, “A paciente foi orientada”.

O tempo futuro do presente, em português, pode ser construído por meio de uma desinência (por ex. “á” para a terceira pessoa do singular: “soltará”, “fará”, “motivará”) e anotado com a *feature* morfológica de futuro. Também pode ser construído por uma perífrase verbal (e.g. “vai soltar”, “vai fazer”, “vai motivar”). Neste segundo caso, nenhum dos verbos recebe a *feature* morfológica de futuro, já que o significado de futuro se dá pela combinação dos dois verbos, e não por uma desinência. As formas construídas por meio de desinência são chamadas de futuro sintético e constroem a noção de futuro com recursos da morfologia. Já as formas construídas por meio de perífrase são denominadas futuro analítico e constroem a noção de futuro com recursos da sintaxe.

Formas negativas de adjetivos podem ser criadas em português por meio de morfemas (e.g. “capaz” x “**incapaz**” ou “normal” x “**anormal**” ou “moral” x “**imoral**”) ou lexicalmente (i.e “governamental” x “**não** governamental”). O processo de formação de palavras por meio da adição de prefixos de negação **a-**, **i-**, **im-**, **in-** é estudado dentro da morfologia, ao passo que a construção da negação a partir da inserção de uma outra palavra é estudada no escopo da sintaxe.

As formas verbais que indicam imperativo em português possuem a mesma morfologia de subjuntivo. A definição entre um modo verbal e outro depende do contexto sintático em que estão inseridas. Por exemplo na sentença “Faça isso imediatamente”, o verbo “faça” é classificado como modo imperativo. Já na sentença “Quero que você faça isso imediatamente”, o verbo “faça” é classificado como subjuntivo. Para definir a *feature* morfológica de modo verbal a ser atribuída a formas como “faça” é preciso antes fazer a análise sintática da sentença.

6.8.2 Sintaxe e Semântica

A análise sintática pode ser complementada pela análise no nível da semântica. Essa complementação pode ser ilustrada com o seguinte exemplo.

Dada uma oração como “João deu um pulo”, uma análise sintática de constituintes nos permite identificar dois constituintes maiores: “João” e “deu um pulo”, este último passível de ser segmentado em constituintes menores contidos nele: “deu” e “um pulo”. Numa análise de sintaxe de dependência, observamos as relações entre as palavras e identificamos “deu” como sendo a palavra raiz (**root**), “João” seu dependente em relação de sujeito (**nsubj**) e “pulo” em relação de objeto (**obj**). Assim, interpretamos que alguém (“João”) realiza uma ação (“dar”) que tem um objeto (“um pulo”). No entanto, todos nós interpretamos “dar um pulo” como expressão de uma ação equivalente a “pular”. Como vimos neste capítulo, os constituintes que exercem a função de objeto geralmente correspondem a um papel



temático na semântica. Em uma oração como “João deu um dinheiro”, “dinheiro” é objeto do verbo “dar” e cumpre o papel temático de objeto estativo. Já em “João deu um pulo”, o objeto “pulo” não possui status de papel temático; portanto, consideramos “deu um pulo” como uma única unidade do ponto de vista semântico. Assim, a análise sintática de dependência apresenta a mesma configuração para orações como “João deu um pulo” e “João deu um dinheiro”. Em PLN, casos como este demandam uma análise semântica complementar à análise sintática, pois a identificação de papéis temáticos é relevante em tarefas como a extração de informação.

6.8.3 Sintaxe e Pragmática

A análise sintática pode, também, ser complementada pela análise sob a perspectiva da pragmática. Os exemplos a seguir ilustram essa complementação.

Em português, temos configurações de sentenças com uma palavra indicando negação, as quais constroem, no entanto, significados positivos. Numa sentença exclamativa como “Quantas vezes ela não ligou chorando!”, temos o advérbio “não”, que constrói um significado negativo. Contudo, interpretamos a exclamação como construindo um significado afirmativo: “ela ligou chorando muitas vezes”. Em casos como este, de sentenças com elementos negativos que constroem um significado afirmativo, vemos que a perspectiva pragmática, isto é, a análise da linguagem em uso e dos pressupostos e inferências que fazemos como falantes, é relevante para complementar a análise sintática. Do contrário, interpretaremos a exclamação como a negação do evento: “ela não ligou chorando”.

A perspectiva da pragmática também nos ajuda a interpretar sentenças nas quais não há elemento de negação, mas o significado construído é negativo. Por exemplo, em “Pessoa alguma será condenada”, não temos nenhuma palavra que construa o significado de negação. No entanto, a oração é interpretada como “Nenhuma pessoa será condenada”. Cabe destacar que, se trocarmos a ordem das palavras neste exemplo, teremos uma interpretação distinta, como é evidente ao contrastarmos “Pessoa alguma será condenada” e “Alguma pessoa será condenada”.

6.8.4 Sintaxe e Discurso

O nível do Discurso e os Modelos Discursivos (Capítulo [Modelos discursivos](#)) serão apresentados mais à frente neste livro, mas convém explicitar neste momento uma confusão que se faz comumente entre os níveis sintático e discursivo.

A Gramática Tradicional (GT), nas seções referentes à sintaxe, estuda os conceitos de frase, oração e período, assim como a classificação das orações dentro de um período, por exemplo em oração principal, oração coordenada adversativa ou oração subordinada adverbial consecutiva, dentre várias outras.

Essas mesmas relações são estudadas em PLN no nível do Discurso como relações de contraste, de explicação, de equivalência, dentre várias outras. Isso porque as teorias discursivas, tais como RST (*Rhetorical Structure Theory*) e CST (*Cross-document Structure Theory*) não limitam sua unidade de análise linguística apenas dentro do texto, da sentença, do período ou da frase. São teorias que estudam o discurso independente do tamanho ou dos limites marcados por sinais de pontuação.

Apesar do rol de relações das teorias discursivas não representar uma equivalência exata com os tipos de orações da Gramática Tradicional, podemos identificar paralelos, por exemplo, entre uma relação de *Summary* (da RST) e uma oração coordenada conclusiva



(da GT) ou entre uma relação discursiva de *Indirect Speech* (da CST) com uma oração subordinada substantiva objetiva direta).

6.9 Considerações finais

Neste capítulo, introduzimos os conceitos básicos de sintaxe, que é a área da linguística responsável por definir e descrever a ordem e a função das palavras e sintagmas na frase. Assim, foram discutidos conceitos linguísticos de **classe de palavra**, **constituente**, **sintagma**, **frase**, **oração**, **período** (ou **sentença**), **escala de ordens**, **função sintática**, **papel temático**, assim como conceitos sintáticos voltados para o PLN, tais como *part-of-speech* (PoS), *parser*, *parsing*, *treebanks*, árvore de dependência, núcleo (*head*, *governor*, *parent*), dependente (*child*, *dependent*), entre outros.

Como o escopo deste livro é o processamento de língua natural, explicitamos as duas principais abordagens sintáticas usadas em PLN, que são: a **sintaxe de constituintes** e a **sintaxe de dependência**. Mas é importante ressaltar que existem várias correntes linguísticas teóricas que estudam e descrevem a sintaxe a partir de diferentes pontos de vista. Algumas dessas correntes são: paradigma formal ou estrutural, paradigma funcional ou funcionalista, o sistêmico-funcional, o gerativo ou gerativista, dentre outros.

Tanto a sintaxe de constituintes quanto a de dependência são abordagens amplamente usadas em PLN e, para isso, precisam de uma representação formal. Foram apresentados os tipos mais comuns de representação dessas duas abordagens, a saber: por meio de **colchetes**, de **árvores**, de **setas**, de **parênteses** e de **indentação**.

Ao final do capítulo, apresentamos alguns exemplos de questões e problemas linguísticos que podem ser estudados pelo viés da sintaxe ou de algum outro nível linguístico, já que essas fronteiras não são tão bem definidas na língua em uso. Como o foco do livro está na língua portuguesa, nos limitamos a mencionar brevemente alguns poucos exemplos. Para um estudo mais aprofundado sobre as fronteiras entre a sintaxe e os demais níveis linguísticos, ver Bender (2013).

O próximo capítulo (Capítulo **Ferramentas e recursos para o processamento sintático**) também é dedicado ao estudo da sintaxe dentro do PLN, porém com um olhar mais computacional voltado para os recursos e ferramentas disponíveis para fazer análise sintática automaticamente, os tipos de *parsing* e as abordagens sintáticas mais comuns em PLN.

Referências

BENDER, E. M. **Linguistic Fundamentals for Natural Language Processing: 100 Essentials from Morphology and Syntax**. Springer Nature Switzerland AG 2013: Springer Cham, 2013. p. XVII–166

KENEDY, E.; OTHERO, G. DE Á. **Para conhecer sintaxe**. São Paulo: Contexto, 2018.

MCDONALD, R. et al. **Universal Dependency Annotation for Multilingual Parsing**. Proceedings of the 51st Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers). **Anais...**Sofia, Bulgaria: Association for Computational Linguistics, ago. 2013. Disponível em: <<https://aclanthology.org/P13-2017>>

NIVRE, J. et al. **The CoNLL 2007 Shared Task on Dependency Parsing**. Procee-



dings of the 2007 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning (EMNLP-CoNLL). **Anais...**Prague, Czech Republic: Association for Computational Linguistics, jun. 2007. Disponível em: <<https://aclanthology.org/D07-1096>>

OSBORNE, T.; GERDES, K. **The status of function words in dependency grammar: A critique of Universal Dependencies (UD)**. **Glossa: a journal of general linguistics (2016-2021)**, jan. 2019.

RADEMAKER, A. et al. **Universal Dependencies for Portuguese**. Proceedings of the Fourth International Conference on Dependency Linguistics (Depling 2017). **Anais...**Pisa,Italy: Linköping University Electronic Press, set. 2017. Disponível em: <<https://aclanthology.org/W17-6523>>

TESNIÈRE, L. **Eléments de Syntaxe Structurale**. Paris: Klincksieck, 1959.

ZEMAN, D. **Reusable Tagset Conversion Using Tagset Drivers**. Proceedings of the Sixth International Conference on Language Resources and Evaluation (LREC'08). **Anais...**Marrakech, Morocco: European Language Resources Association (ELRA), 2008. Disponível em: <http://www.lrec-conf.org/proceedings/lrec2008/pdf/66_paper.pdf>

ZEMAN, D.; RESNIK, P. **Cross-Language Parser Adaptation between Related Languages**. Proceedings of the IJCNLP-08 Workshop on NLP for Less Privileged Languages. **Anais...**2008. Disponível em: <<https://aclanthology.org/I08-3008>>



Capítulo 7

Ferramentas e recursos para o processamento sintático

Elisa Terumi Rubel Schneider

Adriana S Pagano

Ana Clara S Pagano

Lucelene Lopes

Publicado em: 26/09/2023

Atualizado em: 20/11/2024

 <https://brasileiraspln.ufscar.br/livro-pln-4ed-vol1/>

7.1 Introdução

A sintaxe é o nível de análise linguística no qual examinamos os padrões de estruturação de sentenças. Isto é, analisamos como as palavras se organizam em unidades que constroem significado dentro da sentença. Para isso, consideramos a classe de cada palavra, sua ordem na sentença e sua relação com as outras palavras. Conforme visto no Capítulo [A ordem e a função das palavras em uma sentença](#), em PLN, a análise computacional realizada no nível sintático é denominada *parsing*, a ferramenta que realiza essa tarefa é denominada *parser* e o recurso criado por meio da análise sintática é chamado *treebank*.

Neste Capítulo, vamos conhecer tipos de *parsing* sob a perspectiva computacional, juntamente com ferramentas e recursos disponíveis para o processamento do português brasileiro.

7.2 Tipos de *parsing*

A tarefa de *parsing* consiste em, dada uma entrada com uma sentença sem nenhuma anotação (*raw*), um modelo faz uma predição da estrutura sintática dessa sentença. Como vimos no Capítulo [A ordem e a função das palavras em uma sentença](#), o objetivo do processamento sintático é identificar as unidades (como palavras, sintagmas e orações) na sentença e estabelecer as relações gramaticais entre elas a fim de extrair algum tipo de informação. Essas relações podem ser analisadas em termos de:

1. **constituência**, ou seja, quais unidades são hierarquicamente inferiores às outras e estão nelas contidas; ou
2. **dependência**, isto é, quais palavras dependem de quais outras e qual é o tipo de relação entre elas, incluindo o papel de cada palavra na sentença (como sujeito, objeto, verbo, adjetivo etc.).

Assim, de acordo com o tipo de análise sintática adotada, há *parsers* de constituição e *parsers* de dependência.



Mas há uma perspectiva adicional sob a qual podemos caracterizar tipos de *parsing* e *parsers*: trata-se do escopo ou profundidade com que a análise sintática é executada. Nesse sentido, podemos analisar as sentenças de forma exaustiva até obtermos uma análise completa de sua estrutura ou fazer uma análise mais rasa para obtermos uma análise com informações mínimas, mas relevantes para as tarefas em PLN.

O primeiro tipo é denominado *deep* (em português, *profundo*) ou *parsing* completo e o segundo tipo é denominado *shallow* (em português, *superficial*) ou *parsing* parcial. Contudo, cabe uma observação sobre esta terminologia. No uso geral, os termos *parsing* e *parser* acabaram sendo adotados para se referir ao *parsing* completo. Já o *parsing* parcial é conhecido como *chunking* (em português, *cortar*) e a ferramenta como *chunker*, embora *chunking* seja uma dentre várias abordagens para a implementação do *parsing* parcial (Jurafsky; Martin, 2023).

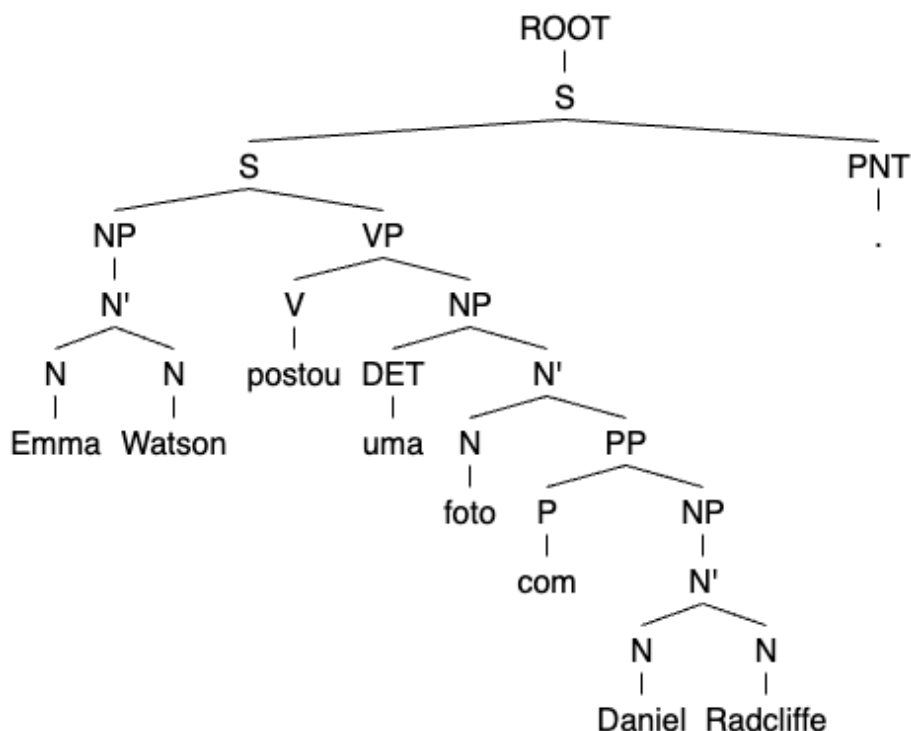
Tanto o *parsing* de constituintes como o *parsing* de dependência podem ser executados de forma completa ou parcial. Tomando como exemplo o *parsing* de constituintes, uma análise completa ou *deep* extrai todos os agrupamentos e as relações sintáticas em uma sentença. Por exemplo, dada a sentença:

Exemplo 7.1:

Emma Watson postou uma foto com Daniel Radcliffe.

Temos na Figura 7.1 uma representação em diagrama de árvore que mostra a profundidade da análise.

Figura 7.1: Exemplo de saída de *parser*



Já uma análise parcial extrai constituintes delimitados, sem estabelecer a hierarquia entre eles ou de que forma uns estão contidos em outros.

Figura 7.2: Exemplo de saída de *parser* parcial

[NP Emma Watson] [VP postou] [NP uma foto] [PP com] [NP Daniel Radcliffe]
--

A Figura 7.2 ilustra a análise rasa, não hierárquica do *parsing* parcial para o Exemplo 7.1. O objetivo do *parsing* parcial é gerar uma representação rasa da estrutura da sentença que possibilite um processamento mais rápido de grandes volumes de texto. É geralmente implementado por meio de tokenização de uma sentença em palavras, identificação da classe de palavra (PoS) e segmentação em pedaços ou *chunks*. O conceito de *chunk* foi proposto por Abney (1992) como uma unidade formada por uma única palavra ou por um conjunto de palavras. Em um *chunk*, há uma palavra de conteúdo circundada por palavras funcionais. A palavra de conteúdo mais explorada em *chunking* é o substantivo, dada a alta correlação de substantivos com entidades.

Assim, a tarefa de *chunking* da sentença do Exemplo 7.1, executada em Python utilizando o modelo de língua portuguesa `pt_core_news_sm`, da biblioteca `spaCy`, gera o resultado disposto na Figura 7.3:

Figura 7.3: Captura de tela de *notebook* em Python com código e resultado de *chunking* de uma sentença em português

```

1  !pip install spacy
2  !python -m spacy download pt_core_news_sm
3  import spacy
4  nlp = spacy.load("pt_core_news_sm")
5  sentence = "Emma Watson postou uma foto com Daniel Radcliffe."
6  doc = nlp(sentence)
7  for chunk in doc.noun_chunks:
8      print(chunk.text)

```

```

Emma Watson
uma foto
Daniel Radcliffe

```

Como vemos na Figura 7.3, o *chunking* reconhece três unidades ou *chunks*, cada uma nucleada por um substantivo. Os três *chunks* são candidatos a entidades, sendo duas delas nomes próprios de pessoas. De fato, como veremos no Capítulo [Extração de Informação](#), vários modelos de Extração de Informação utilizam análises rasas como a fornecida pelo *chunking*.

7.3 Recursos e ferramentas para o português

Nesta seção, vamos conhecer alguns recursos e ferramentas de PLN para análise sintática do português.



7.3.1 Corpora

O primeiro recurso para o processamento linguístico é um *corpus* anotado ou *treebank*, isto é, textos enriquecidos com marcações de classe de palavras (*Part-of-Speech*) e relações sintáticas. Um exemplo de *corpus* em português anotado é o Bosque¹, amplamente utilizado para treinar modelos de análise sintática (Veja Capítulo [Conjunto de dados, dataset e corpus](#)).

O *corpus* Bosque é parte de um *corpus* maior, chamado Floresta Sintá(c)tica², que abrange, além do Bosque, outros *subcorpora*, nomeadamente: Selva, Amazônia e Floresta Virgem. O grande *corpus* foi anotado automaticamente pelo *parser* PALAVRAS (Bick, 2000). O Bosque está integrado por sentenças extraídas dos *corpora* CETENFolha (português brasileiro) e CETEMPúblico (português europeu), ambos constituídos por textos jornalísticos escritos. Uma versão do Bosque³ foi convertida para o formato UD (Universal Dependencies), apresentado no Capítulo [A ordem e a função das palavras em uma sentença](#), e é hoje um dos *treebanks* mais utilizados pela comunidade de PLN no Brasil em modelos de *parsing* de dependência atuais.

Além da Floresta Sintá(c)tica, encontra-se disponível, como recurso para a língua portuguesa, o Corpus Internacional do Português – CINTIL⁴, desenvolvido pela Universidade de Lisboa, que possui 1 milhão de *tokens* de texto jornalístico, com anotação de classe de palavra, lema e expressões multipalavra. Uma versão desse *corpus*, o CINTIL-UDep⁵, é disponibilizada com anotações no padrão UD.

Mais recentemente, o *corpus* PetroGold⁶ foi disponibilizado e, hoje, é um *corpus* passível de ser utilizado em modelos de *parsing* de dependência. PetroGold é um *corpus* de textos acadêmicos no domínio do petróleo, anotado no formato UD e revisado manualmente.

Há diversas iniciativas em andamento, no momento da escrita deste capítulo, para a criação de *corpora* anotados em português brasileiro. No escopo do projeto NLP2, desenvolvido pelo Centro de Inteligência Artificial⁷ (C4A1) da Universidade de São Paulo, com o objetivo de desenvolver recursos, ferramentas e aplicações para levar o português ao estado da arte em PLN, o projeto POeTiSA⁸ desenvolve o *treebank* Porttinari⁹, um *corpus* multi-gênero de textos em português brasileiro anotados de acordo com o padrão UD. Inclui textos jornalísticos do *corpus* da Folha de São Paulo/Kaggle, o *corpus* MAC-MORPHO¹⁰ de textos jornalísticos, o *corpus* DANTE¹¹ (*Dependency-ANalised corpora of TwEets*), integrado por tweets da Bolsa de Valores, B2W-reviews01¹², composto por resenhas e avaliações de consumidores da empresa de comércio eletrônico Americanas e um *corpus* de Resenhas online de livros. A versão Porttinari-base já se encontra disponível¹³.

Uma iniciativa também em andamento é o *corpus* Veredas¹⁴, desenvolvido na Faculdade

¹<https://www.linguateca.pt/Floresta/corpus.html>

²<https://www.linguateca.pt/Floresta/principal.html>

³https://universaldependencies.org/treebanks/pt_bosque/index.html

⁴<http://cintil.ul.pt/>

⁵https://universaldependencies.org/treebanks/pt_cintil/index.html

⁶https://universaldependencies.org/treebanks/pt_petrogold/index.html

⁷<https://c4ai.inova.usp.br/>

⁸<https://sites.google.com/icmc.usp.br/poetisa/the-project>

⁹<https://sites.google.com/icmc.usp.br/poetisa/resources-and-tools>

¹⁰Mac-Morpho: <http://www.nilc.icmc.usp.br/macmorpho/>

¹¹Brazilian Stock Market Tweets with Emotions|Kaggle: <https://www.kaggle.com/datasets/fernandojvdasilva/stock-tweets-ptbr-emotions>

¹²<https://opencor.gitlab.io/corpora/real19b2wreviews01/>

¹³Corpus Porttinari: <https://sites.google.com/icmc.usp.br/poetisa/porttinari>

¹⁴<http://www.lettras.ufmg.br/veredas/>



de Letras da UFMG, que visa à construção de *treebanks* de textos anotados de acordo com o padrão das UD. Inclui amostras de uma variedade de textos em inglês, espanhol e português brasileiro: colunas jornalísticas, fábulas, narrativas, receitas culinárias, questionários médicos e bulas de medicamento. Em parceria com a PUCPR, a Faculdade de Letras da UFMG desenvolveu o *treebank* DepClinBr, um *corpus* de narrativas clínicas anotadas de acordo com o padrão das UD (Oliveira et al., 2022).

7.3.2 Parsers

Parsers são ferramentas que podem auxiliar uma aplicação (por exemplo, tradução automática, sumarização de textos, extração de informação, *question-answering*) ou fazer parte de uma ferramenta maior ou conjunto de ferramentas (*toolkit*).

Algumas ferramentas computacionais estão disponíveis para realizar a análise sintática em português. A análise pode ser feita por meio de:

7.3.2.1 Programas e aplicativos

Existem diversos *parsers* desenvolvidos por distintos grupos de pesquisa, fornecidos através de um programa de computador ou aplicativo a ser instalado. No Brasil, podemos citar Curupira¹⁵, Donatus¹⁶ e PassPort¹⁷. Curupira é um analisador robusto de uso geral para o português brasileiro, fornecendo um conjunto das análises sintáticas possíveis para uma frase de entrada. A ferramenta analisa sentenças de cima para baixo, da esquerda para a direita, por meio de uma gramática funcional livre de contexto, restrita e relaxada, para o português brasileiro escrito padrão e um léxico extenso e de ampla cobertura. A Figura 7.4 apresenta a interface gráfica onde é possível ver a obtenção de toda informação da análise realizada pelas regras do *parser*¹⁸.

Donatus é um projeto que consiste em ferramentas e gramáticas baseadas em Python e na biblioteca NLTK¹⁹ para análise profunda e anotação sintática de *corpora* do português brasileiro. Inclui uma interface gráfica, conforme pode ser visto na Figura 7.5. Está disponível em repositório público, sob licença *GNU General Public License version 3.0 (GPLv3)*.

PassPort é uma ferramenta para análise de dependências de português treinado com o Stanford Parser, utilizando o *corpus* Portuguese Universal Dependency (PT-UD). Infelizmente, a página do projeto não se encontra disponível na data de escrita deste capítulo.

7.3.2.2 Frameworks e Bibliotecas

Devido à popularidade de Python, muitas bibliotecas de PLN foram desenvolvidas na linguagem. Entre as bibliotecas que incluem *parsing* para língua portuguesa, podemos citar spaCy²⁰, Stanza²¹ e NLTK.

¹⁵<http://www.nilc.icmc.usp.br/nilc/tools/curupira.html>

¹⁶<https://sourceforge.net/projects/donatus/>

¹⁷PassPort (A Dependency Parsing Model for Portuguese | SpringerLink): https://link.springer.com/chapter/10.1007/978-3-319-99722-3_48

¹⁸No entanto, vale ressaltar que o projeto foi desenvolvido em 2002 - 2004, de acordo com o site, e pode não estar mais disponível para ser obtido.

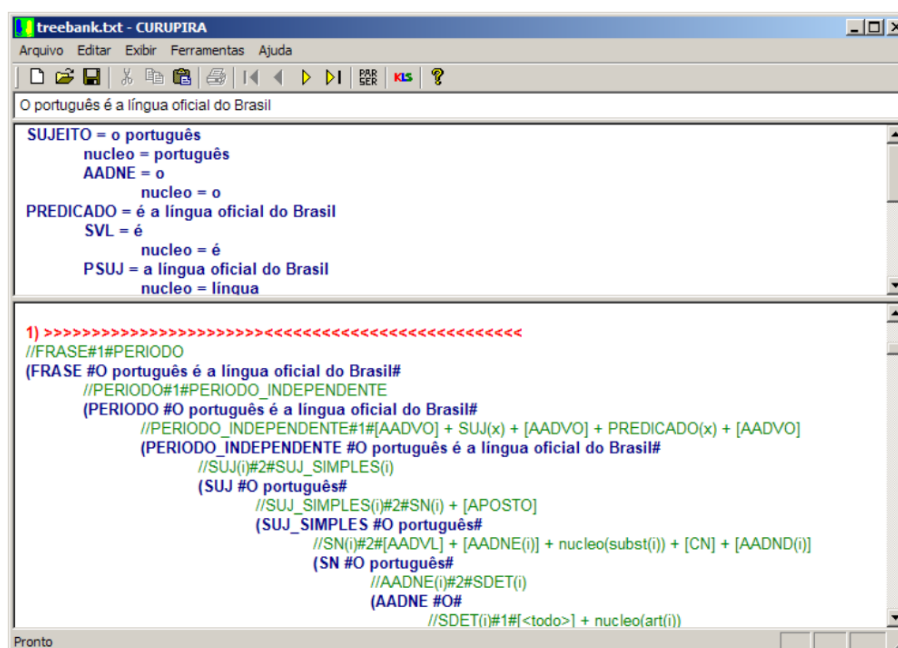
¹⁹NLTK :: *Natural Language Toolkit* <https://www.nltk.org>

²⁰<https://spacy.io/>

²¹<https://stanfordnlp.github.io/stanza/index.html>



Figura 7.4: Captura de tela da interface gráfica da ferramenta Curupira



Fonte: (Martins et al., 2003)

spaCy é uma biblioteca PLN que oferece análise linguística eficiente e rápida para várias línguas, incluindo o português. Inclui recursos para tokenização, marcação de parte do discurso (PoS *tagging*), reconhecimento de entidades nomeadas, análise sintática e outros. Através de modelos pré-treinados, o spaCy é capaz de fornecer análises detalhadas, permitindo a extração de informações semânticas de um texto em língua portuguesa.

Stanza²² é outra biblioteca PLN que suporta vários idiomas, incluindo o português, desenvolvida pela Universidade Stanford. Fornece uma gama de recursos semelhantes ao spaCy, com suporte a análises mais profundas, como a análise de dependência neural.

NLTK (*Natural Language Toolkit*) também é uma biblioteca em Python que oferece suporte para tarefas de PLN em língua portuguesa, como a análise sintática. NLTK permite o *parse* usando expressões regulares (com *Regex Parser*), análise de dependência com analisador de dependência probabilístico e análise de dependência com analisador de Stanford.

É importante mencionar que, além de Python, outras linguagens de programação também oferecem bibliotecas e *frameworks* para análise sintática.

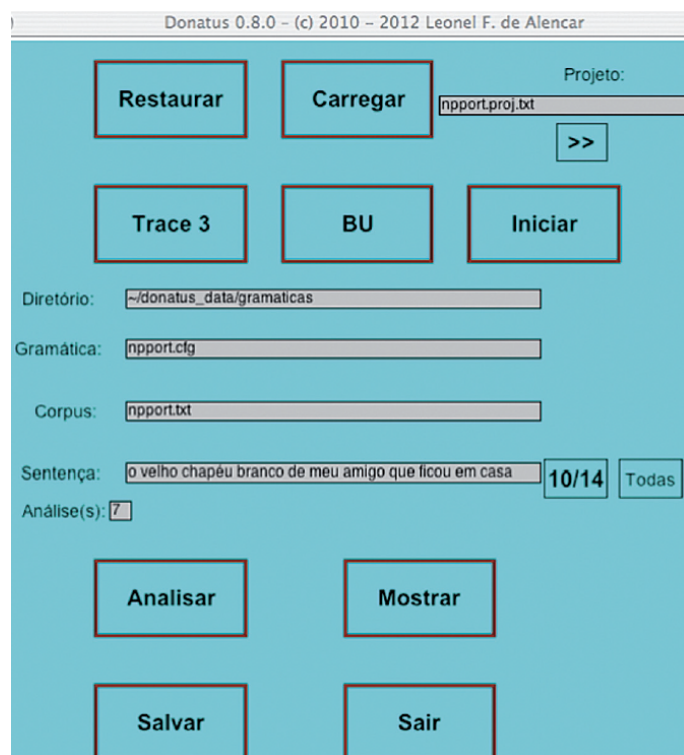
7.3.2.3 Ferramentas online

A seguir apresentaremos algumas das ferramentas web disponíveis para *parsing* de texto em português, permitindo executar a análise sintática e obter como saída arquivos com distintos formatos, incluindo a visualização das árvores sintáticas. Todas as ferramentas apresentadas são de acesso livre e gratuito.

O Portparser

²²Cabe esclarecer que embora o Stanford forneça um modelo treinado para língua portuguesa, que pode ser utilizado pelas bibliotecas Python, na interface online deste projeto não há suporte para o idioma português.

Figura 7.5: Captura de tela da interface gráfica da ferramenta Donatus



Fonte: (Alencar, 2012)

O Portparser (Lopes; Pardo, 2024) é um modelo de parsing para o português brasileiro que se alinha à iniciativa de Universal Dependencies (UD), tendo sido desenvolvido no contexto do projeto POeTiSA (PORTuguese processing - Towards Syntactic Analysis and parsing)²³. O modelo foi treinado com o subcorpus jornalístico Porttinari-base (Duran et al., 2023) do treebank Porttinari (Pardo et al., 2021). O Portparser faz anotação de Part-of-Speech (PoS), identificação de lemas, extração de características morfológicas e anotação de relações de dependência.

O desempenho para o português é de 99% de precisão para anotação de PoS, lema e atributos morfológicos, enquanto a precisão para a anotação de dependências supera 95% de LAS (Labeled Attachment Score), que é a métrica mais tradicional da tarefa de parsing (Nivre; Fang, 2017), sendo o estado da arte conhecido no momento para a língua portuguesa.

A ferramenta que permite o uso do Portparser está disponível online²⁴. A Figura 7.6 apresenta a captura da tela de entrada do Portparser. O Portparser foi projetado para receber textos em português e executar segmentação sentencial, tokenização e as anotações do modelo citadas (PoS, lema, características morfológicas e relações de dependência).

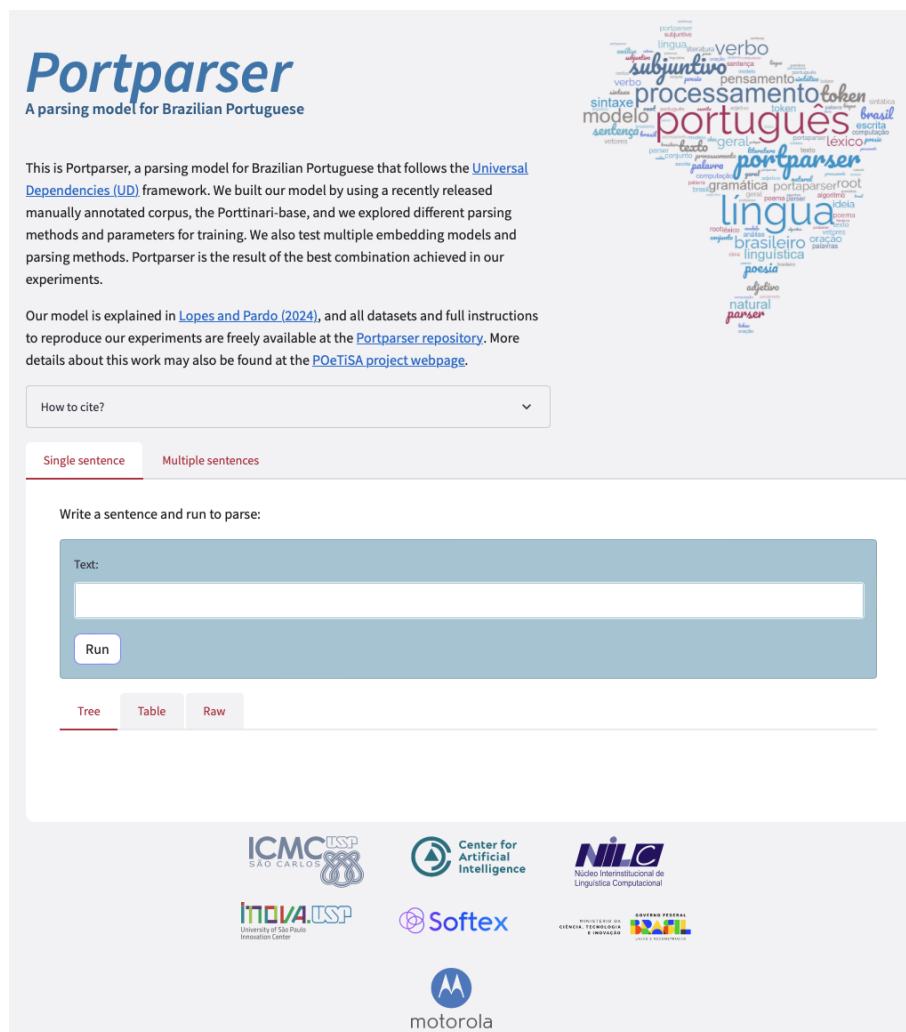
O Portparser aceita entrada de sentenças isoladas (Single sentence) ou upload de arquivo com diversas sentenças (Multiple sentences). No caso de entrada de um arquivo com diversas sentenças, o Portparser oferece a possibilidade de executar a segmentação sentencial do texto ou considerar que o texto já foi segmentado e possui uma sentença por linha.

O resultado da utilização do Portparser pode ser visto na Figura 7.7, onde a sentença

²³<https://sites.google.com/icmc.usp.br/poetisa>

²⁴<http://portparser.icmc.usp.br:8082/>

Figura 7.6: Captura da tela de entrada da ferramenta online Portparser



“Gostei da variedade e da qualidade” foi anotada. A anotação produzida pode ser visualizada através da árvore de dependência (Tree), como na Figura 7.7, mas também por uma tabela com o conteúdo do arquivo CoNLL-U (Table) na Figura 7.8 ou como um texto puro com o arquivo CoNLL-U (Raw) na Figura 7.9.

Figura 7.7: Execução do Portparser para a sentença “Gostei da variedade e da qualidade.” visualizando a árvore de dependência gerada.

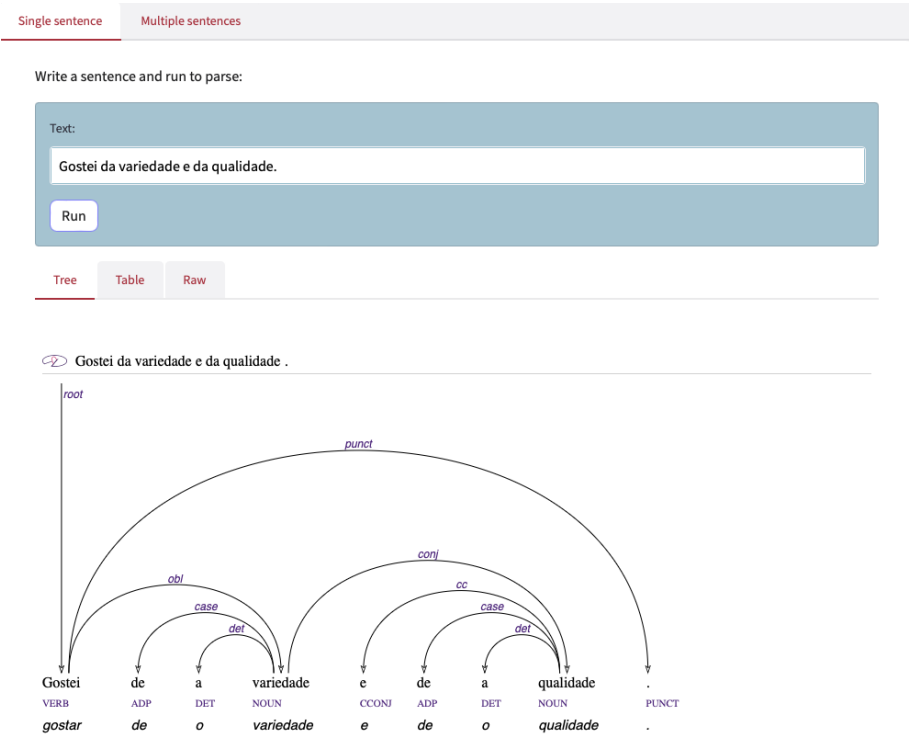


Figura 7.8: Execução do Portparser para a sentença “Gostei da variedade e da qualidade.” visualizando os campos do formato CoNLL-U gerado em forma de tabela.

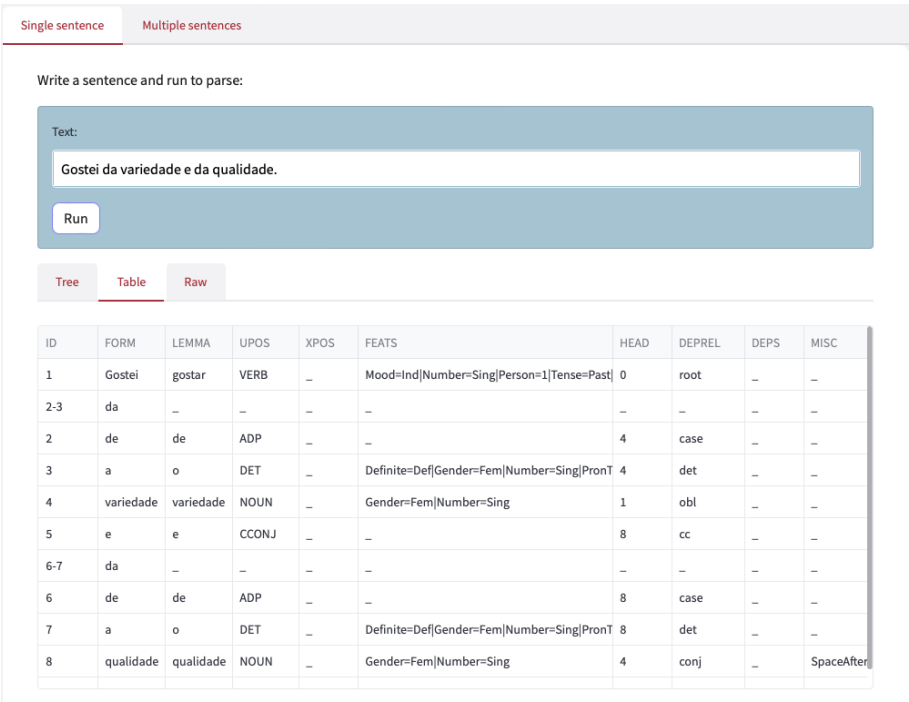


Figura 7.9: Execução do Portparser para a sentença “Gostei da variedade e da qualidade.” visualizando os campos do formato CoNLL-U gerado em forma de texto puro.

Single sentenceMultiple sentences

Write a sentence and run to parse:

Text:

Gostei da variedade e da qualidade.

Run

TreeTableRaw

newdoc id = /tmp/tmp706ile4n/21082024_1154156515253_input.conllu

newpar

sent_id = S0001

text = Gostei da variedade e da qualidade.

1Gostei gostar VERB - Mood=Ind|Number=Sing|Person=1|Tense=Past|VerbForm=Fin0root

2-3da - - - - - - - -

2de de ADP - - 4case - -

3a o DET - - Definite=Def|Gender=Fem|Number=Sing|PronType=Art4det

4variedade variedade NOUN - Gender=Fem|Number=Sing1obl - -

5e e CCONJ - - 8cc - -

6-7da - - - - - - - -

6de de ADP - - 8case - -

7a o DET - - Definite=Def|Gender=Fem|Number=Sing|PronType=Art8det

8qualidade qualidade NOUN - Gender=Fem|Number=Sing4conj - Space/

9. . PUNCT - - 1punct - -

 ©2026 Direitos autorais deste livro a seus autores. Uso permitido sob licença Creative Commons Atribuição-NãoComercial-SemDerivações 3.0 Brasil (CC BY-NC-ND 3.0 BR).

10

O Parser LX

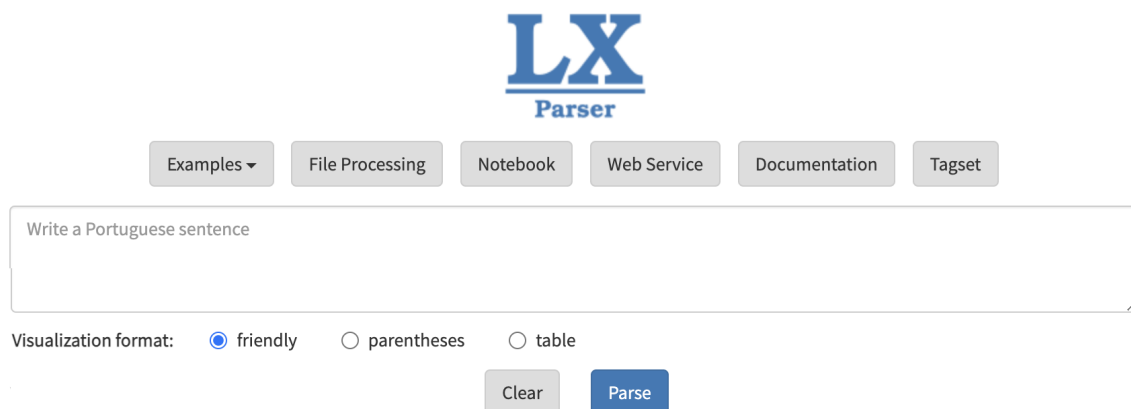
O Parser LX, ferramenta integrante do PORTULAN CLARIN²⁵, é parte integrante de um portal de acesso a infraestrutura de tecnologia linguística no escopo do projeto internacional CLARIN ERIC²⁶.

O Parser LX é disponibilizado tanto para *parsing* de constituição²⁷ como de dependência, este último em duas versões: LX-DepParser²⁸ e LX-UDParser²⁹.

A interface é simples e amigável, tendo como entrada uma sentença que o usuário pode digitar em campo próprio ou um arquivo que deverá ser importado.

A Figura 7.10 mostra uma captura de tela da interface do Parser LX de constituição.

Figura 7.10: Captura de tela mostrando a interface do *parser* de constituição LX.



Como vemos na Figura 7.10, o *parser* tem como saída uma visualização na forma de árvore, denominada “amigável” (em inglês, *friendly*), ou uma representação parentética ou entre parênteses/colchetes ou ainda uma representação na forma tabular.

Para a sentença “Emma Watson postou uma foto com Daniel Radcliffe”, o *parser* gera a árvore de constituição apresentada na Figura 7.11

No que diz respeito à sintaxe de dependência, LX possui duas versões, sendo uma delas adaptada ao formato Universal Dependencies, apresentado no Capítulo A ordem e a função das palavras em uma sentença.

A Figura 7.12 mostra uma captura de tela da interface do Parser LX de dependência no formato UD.

Como a tela mostra, a saída do *parser* pode ser na forma de visualização com setas, denominada “amigável” ou no formato CoNLL-U, que é um formato próprio para *parsing* de dependência, como vimos no Capítulo A ordem e a função das palavras em uma sentença.

Para a sentença “Emma Watson postou uma foto com Daniel Radcliffe”, o *parser* gera a árvore de constituição apresentada na Figura 7.13.

²⁵<https://portulanclarin.net/>

²⁶<https://www.clarin.eu/>

²⁷<https://portulanclarin.net/workbench/lx-parser/>

²⁸<https://portulanclarin.net/workbench/lx-depparser/>

²⁹<https://portulanclarin.net/workbench/lx-udparser/>



Figura 7.11: Diagrama de árvore de constituição gerado pelo *parser* LX.

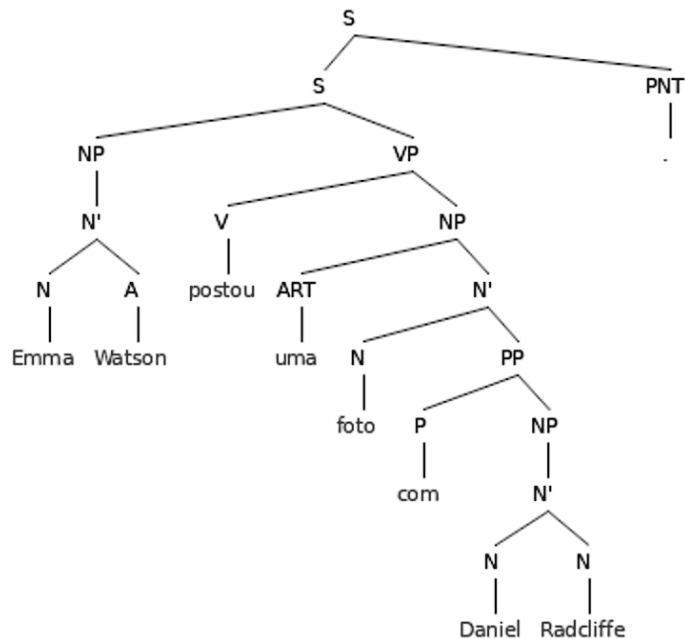


Figura 7.12: Captura de tela mostrando a interface do *parser* de dependência LX.

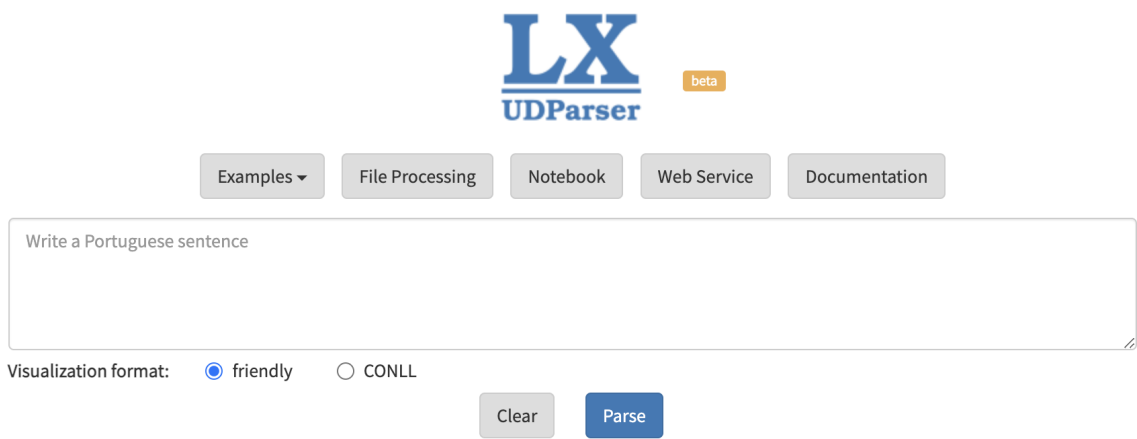
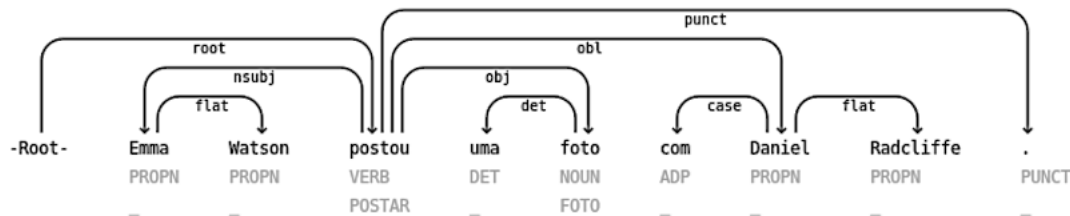


Figura 7.13: Diagrama com setas gerado pelo Parser LX.

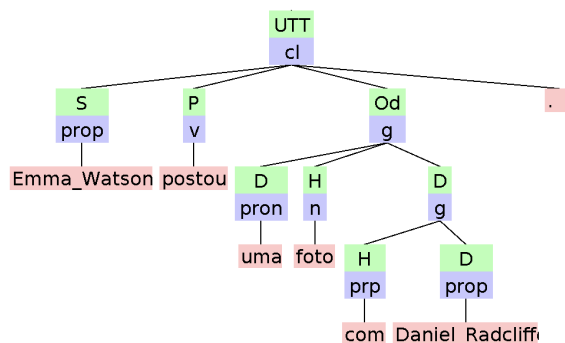


O Parser VISL³⁰

O projeto VISL (do inglês, *Visual Interactive Syntax Learning*) é desenvolvido pelo *Institute of Language and Communication (ISK)* da *University of Southern Denmark*. O projeto disponibiliza recursos (*corpora*) e ferramentas, tais como *parsers* de constituição e dependência. A análise se baseia no *parser* PALAVRAS (Bick, 2000) e no *corpus* Floresta Sintá(c)tica.

Para a sentença “Emma Watson postou uma foto com Daniel Radcliffe”, o *parser* gera a árvore de constituição apresentada na Figura 7.14.

Figura 7.14: Diagrama de árvore de constituição gerado pelo *parser* VISL.



O UDPipe

UDPipe é um conjunto de ferramentas (*toolkit*) e um serviço web que possibilita processar texto por meio de uma *pipeline* que inclui tokenização em palavras, etiquetagem de classe de palavra (PoS), lematização, e *parsing* de dependência. É desenvolvido pelo *Institute of Formal and Applied Linguistics* da *Faculty of Mathematics and Physics* da *Charles University* (República Tcheca). O padrão adotado é das Universal Dependencies e conta com modelos treinados para a maioria dos *treebanks* já anotados no formato UD. Além da interface web, altamente eficiente e amigável, é possível processar texto por meio de um script em Python.

A Figura 7.15 mostra uma captura de tela da interface web do UDPipe, com a seleção do modelo Bosque para a língua portuguesa.

A entrada pode ser texto digitado em um campo próprio ou um arquivo de texto e há três formatos de saída: diagrama arbóreo, formato CoNLL-U e formato tabular.

Para a sentença “Emma Watson postou uma foto com Daniel Radcliffe”, o *parser* gera a árvore de dependência apresentada na Figura 7.16.

³⁰<https://visl.sdu.dk/visl/about/>

Figura 7.15: Captura de tela mostrando a interface do UDPipe.

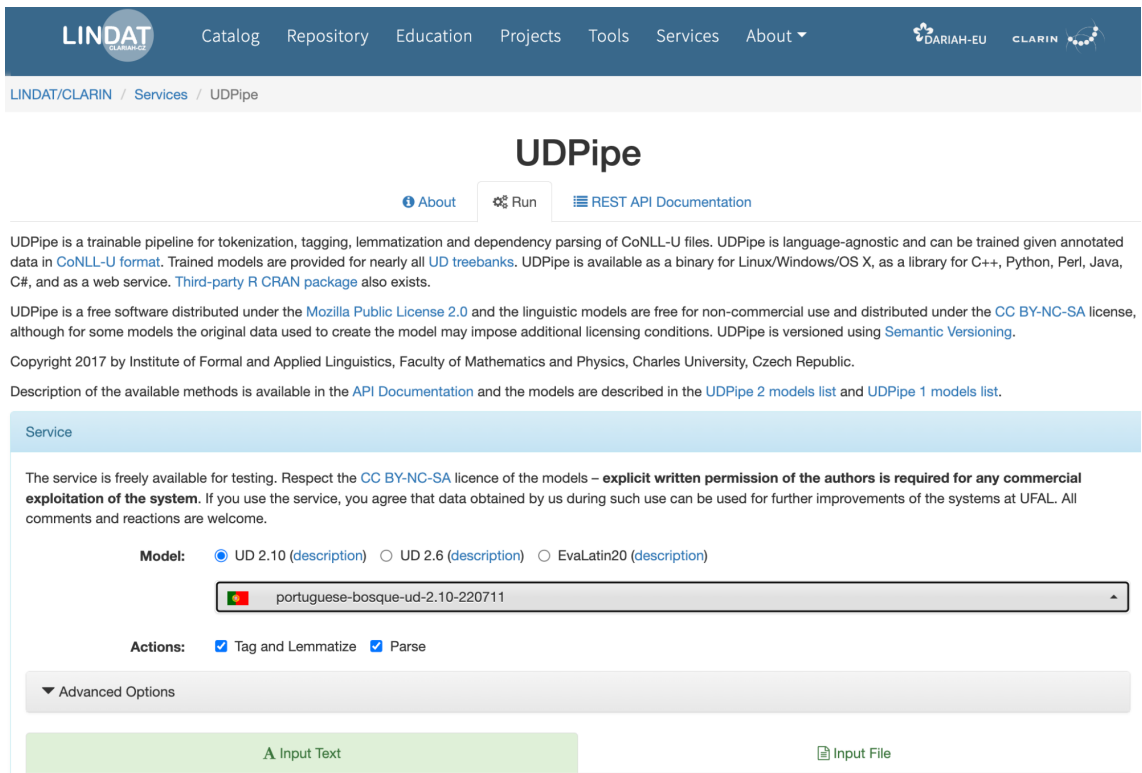
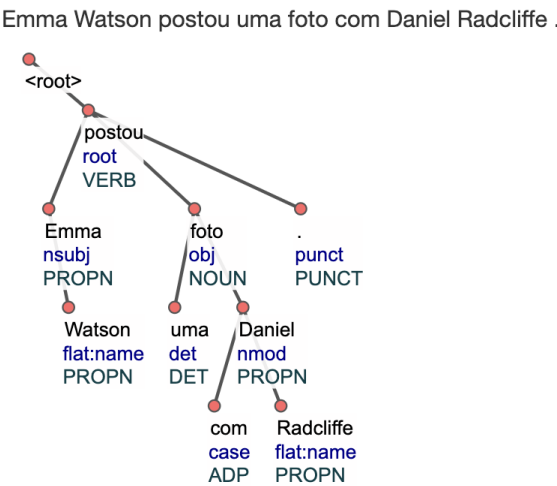


Figura 7.16: Diagrama de árvore de dependência gerado pelo UDPipe.



7.4 Visualização, anotação e edição de *treebanks*

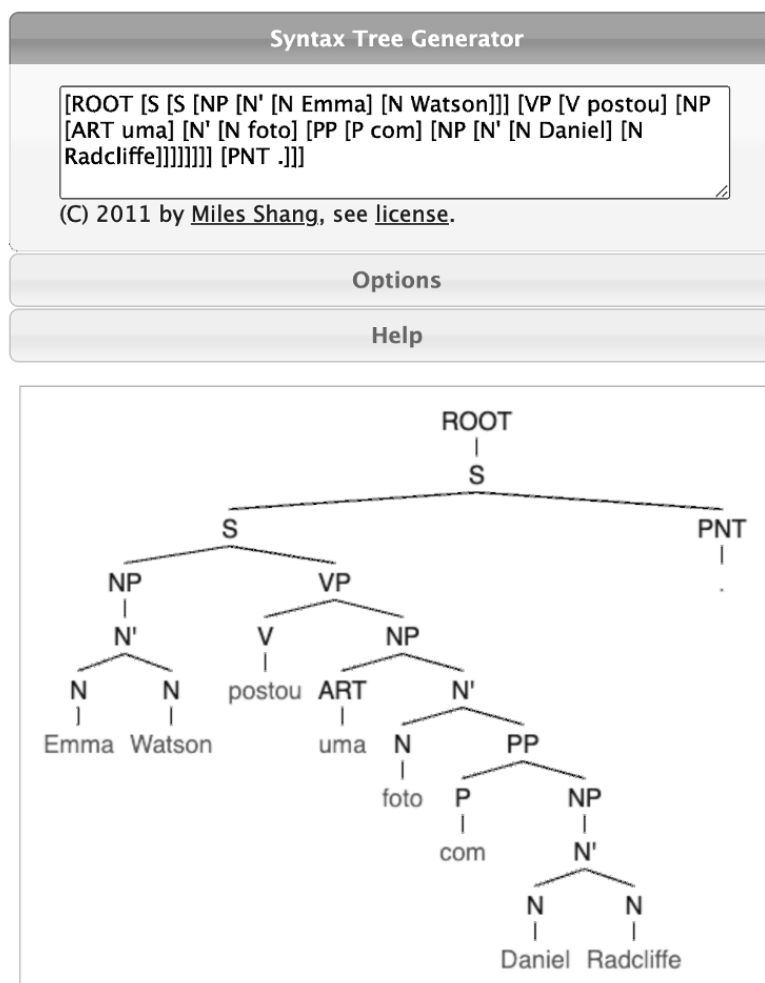
Há diversas ferramentas que permitem tanto a visualização de árvores de constituição e dependência, como a anotação de sentenças e a edição de sentenças já anotadas manualmente ou automaticamente.

7.4.1 Árvores de constituição

Há ferramentas web que oferecem uma visualização gráfica de diagrama de árvores a partir da notação entre colchetes ou parênteses dada como entrada pelo usuário.

Uma dessas ferramentas é *Syntax Tree Generator* (Syntree)³¹, a qual dada uma entrada com notação em colchetes, gera um diagrama de árvore como mostrado na Figura 7.17.

Figura 7.17: Visualização de diagrama de árvore de constituição pela ferramenta Syntree.



Ferramentas semelhantes à Syntax Tree Generator são:

- Jssyntaxtree (*Dynamic JavaScript version of phpSyntaxTree*)³²

³¹<https://mshang.ca/syntree/>

³²<https://ironcreek.net/syntaxtree/>

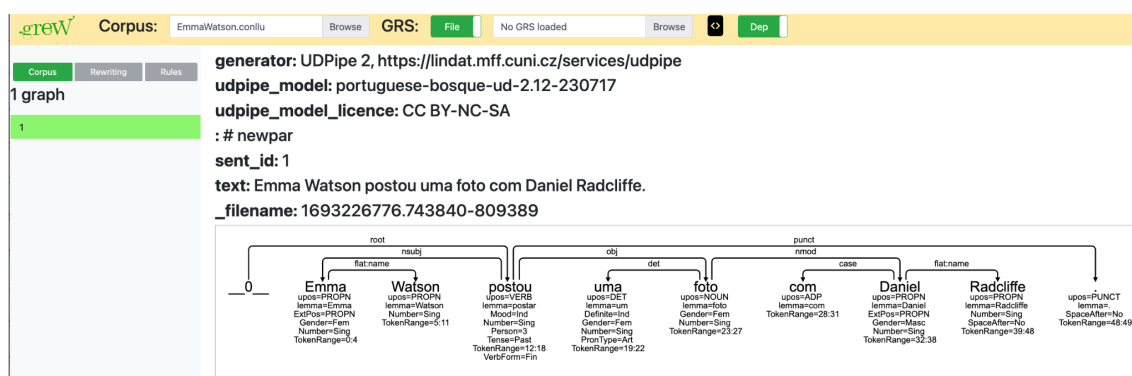
- RSyntaxTree (escrita na linguagem de programação Ruby)³³

7.4.2 Árvore de dependência

7.4.2.1 Visualização de sentenças anotadas

Há ferramentas web que oferecem uma visualização gráfica de diagrama de árvores a partir do arquivo em formato CoNLL-U dado como entrada pelo usuário. Por exemplo, a ferramenta Grew Web possibilita carregar um arquivo CoNLL-U e visualizar o diagrama de relações de dependência, como mostrado na Figura 7.18.

Figura 7.18: Visualização de diagrama de árvore de dependência pela ferramenta Grew Web.



Uma ferramenta semelhante à Grew Web é CoNLL-U Viewer.³⁴

7.4.2.2 Buscas em *treebanks*

Há ferramentas web de busca em *treebanks* anotados com relações de dependência. Esse é o caso da ferramenta Grew Match, que possibilita o acesso a 245 *treebanks* e distintos tipos de busca (por exemplo: por palavra, lema, etiqueta de PoS, etiqueta de relação de dependência, ngramas de palavras, lemas e PoS etc.). A Figura 7.19 mostra uma captura de tela com os resultados de uma busca pela palavra “foto” no *treebank* de dissertações e teses em português brasileiro no domínio do petróleo Petrogold.

Uma ferramenta semelhante à Grew Match é TüNDRA (*Tübingen aNnotated Data Retrieval Application*).³⁵

7.4.2.3 Anotação e edição manual de relações de dependência

Para editar arquivos CoNLL-U previamente anotados de forma manual ou automática, uma das ferramentas mais utilizadas é ArboratorGrew³⁶, que possui uma versão customizada no Brasil pela equipe do ICMC da USP: Arborator-Grew-NILC³⁷. Essa ferramenta permite gerenciar projetos individuais e coletivos de anotação, bem como serve de plataforma instrucional para cursos e treinamentos em anotação de sintaxe de dependência. A

³³<https://yohasebe.com/rsyntaxtree/>

³⁴https://universaldependencies.org/conllu_viewer.html

³⁵<https://weblicht.sfs.uni-tuebingen.de/Tundra/>

³⁶<https://arboratorgrew.elizia.net/#/>

³⁷<https://arborator.icmc.usp.br/#/>